

Lecture 3 - Processes

Operating Systems - Exam Crash Study Guide

Built to help you understand the lecture fast, know what to memorize, and answer exam questions.

Read this first: you do not need to memorize every slide word-for-word. Master the process life cycle, PCB, process states, suspension, creation/termination, and process switching.

Emergency 2-hour plan

Time	What to do	Goal
0-15 min	Process definition, process memory layout, PCB	Know what a process is and what the OS stores about it.
15-45 min	Two-state, five-state, queues, suspended states	Be able to draw/describe state transitions.
45-70 min	Creation and termination, fork(), exec(), wait(), exit(), zombie/orphan	Answer practical UNIX/process questions.
70-95 min	OS control tables, process image, kernel/user mode, process switch	Understand what the OS changes when switching.
95-115 min	UNIX/Linux details: task_struct, UNIX states, UNIX process image	Memorize names and meanings.
Last 5 min	Read the Must Memorize page and flashcards	Lock in the exam vocabulary.

Exam takeaway: Your priority order: PCB -> process states -> creation/termination -> process switching -> UNIX/Linux details.

How to use this guide

The guide follows the lecture in a logical order. Slide page references are included so you can quickly jump back to the PDF when needed.

- Read the short explanation first.
- Then memorize the Exam takeaway box.
- Finally test yourself using the likely questions and flashcards near the end.

Lecture map

Lecture pages	Topic	Why it matters
2-8	Process definition, memory layout, PCB	Core definitions. Very likely exam material.
9-13	Execution, dispatcher, trace, two-state model, queues	Explains how CPU moves between processes.
14-22	Creation, termination, fork/exec/wait, Chrome multiprocess	Practical process examples.
23-33	Five-state model and suspended states	Most diagram-heavy part. Know transitions.
34-48	OS control structures, process image, full PCB details	Explains what OS must store to control resources.
49-59	User/kernel mode, OS kernel functions, process switching, OS as process	Important theory and comparison questions.
60-67	Linux/UNIX representation, UNIX states, UNIX process image, user area	Specific OS examples.

1. What is a process?

A process is a program while it is executing. A program by itself is passive: it is just code stored on disk. A process is active: it has code, data, current execution point, registers, memory, open files, and a current state.

- A process is a sequence of instructions that executes.
- It is the entity assigned to the processor.
- It has associated data needed by the program.
- It has context: all information the OS needs to manage and later resume it.
- It has a current state and system resources.

Exam takeaway: Program = passive file/code. Process = active running instance with state and resources.

Term	Meaning	Example
Program	Stored instructions, not currently executing by itself.	chrome.exe file on disk.
Process	A program in execution with current state and resources.	One running Chrome renderer tab.
Trace	The sequence of instruction addresses executed by a process.	5000, 5001, 5002...
Context	Saved information needed to stop and later continue a process.	PC, registers, state, open files.

2. What the OS must do for processes

The lecture makes a big point: many operating system requirements are expressed in terms of processes.

- Interleave several processes so the CPU is used well while users still get reasonable response time.
- Allocate resources to processes.
- Protect each process from other processes.
- Allow processes to share/exchange information using IPC: InterProcess Communication.
- Synchronize processes so they do not interfere with each other.
- Create and terminate processes.

Exam takeaway: The OS is basically managing many processes safely and fairly at the same time.

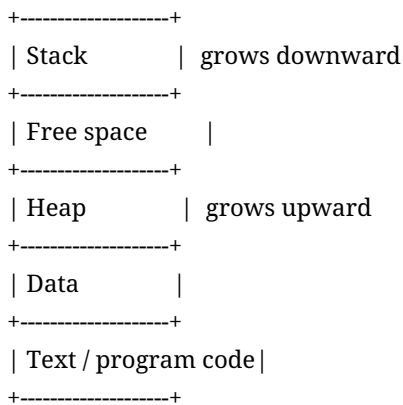
3. Process in memory

A process is not only code. In memory it has multiple regions. This is a favorite exam area because students confuse stack and heap.

Memory part	What it contains	How to remember it
Text section	Program code / machine instructions.	Text = code.
Data section	Global variables and static data.	Data that exists for the whole program.
Current state	Program counter and CPU registers.	Where execution currently is.
Stack	Temporary data: function parameters, return addresses, local variables.	Function calls use the stack.
Heap	Dynamically allocated memory during runtime.	malloc/new/live dynamic objects.

Simple memory picture:

High address



Low address

Exam takeaway: Stack = local temporary function data. Heap = dynamic runtime allocation. Text = code. Data = global variables.

4. Process Control Block (PCB)

The PCB is the data structure the OS creates for every process. It is like the process's ID card plus save file. The OS can interrupt a process, save its information in the PCB, run another process, then restore the first process later.

PCB field	Meaning	Why OS needs it
Process state	Running, ready, blocked, etc.	To know what can run now.
Process number / PID	Unique identifier.	To find and manage the process.
Program counter	Address of next instruction.	To resume at the exact point.
CPU registers	Saved register values.	To restore calculations/state.
Scheduling info	Priority and queue pointers.	To choose the next process.
Memory info	Memory limits, page/segment table pointers.	To find/protect address space.
Accounting info	CPU time, elapsed time, time limits.	For scheduling, billing, monitoring.
I/O status info	Open files and allocated I/O devices.	To continue I/O correctly.

Exam takeaway: If you can explain the PCB, you can explain process switching, queues, and OS process management.

Full PCB categories

Category	Includes	Purpose
Process identification	Process ID, parent process ID, user ID.	Identify ownership and relationships.

Processor state information	User-visible registers, control/status registers, program counter, PSW, stack pointers.	Restore CPU state after interruption.
Processor control information	Scheduling/state info, events, queue links, IPC info, privileges, memory management, resources.	Control and coordinate active processes.

5. Process execution, trace, and dispatcher

The CPU can execute only one instruction stream at a time per core, but the OS switches between processes so it looks like many processes are progressing.

- A dispatcher is a small OS program that switches the processor from one process to another.
- A process trace is the list of instruction addresses executed by a process.
- From the processor's point of view, execution is interleaved: some instructions from Process A, then dispatcher code, then Process B, then dispatcher code, etc.
- Timeout means the running process used up its time slice, so the OS may move it away from the CPU.
- I/O request means the running process may block until the I/O event finishes.

Mini story:

Process A runs a few instructions. A clock interrupt happens. The dispatcher saves A's context, selects Process B, restores B's context, and B runs. If B requests I/O, B becomes Blocked and the dispatcher picks another Ready process.

Exam takeaway: Dispatcher = the switching mechanism. Trace = the actual order of instructions executed.

6. Two-state process model

The simplest model has only two states: Running and Not-running. It is useful as an introduction, but it is too simple because it does not separate processes that are ready from processes waiting for I/O.

State	Meaning
Running	Currently executing on the processor.
Not-running	Not currently executing. It may be ready or waiting, but the two-state model does not separate them.

Enter -> Not-running -> Dispatch -> Running -> Exit

Running -> Pause/timeout -> Not-running

Exam takeaway: Two-state model is too broad because Not-running mixes Ready and Blocked.

7. Five-state process model

The five-state model fixes the problem by splitting Not-running into Ready and Blocked/Waiting. This is one of the most important diagrams in the lecture.

State	Meaning	Can it run immediately?
New	Created but not yet admitted to the executable process pool.	No
Ready	Ready to execute when CPU is given.	Yes
Running	Currently executing.	Already running
Blocked / Waiting	Cannot execute until an event occurs, usually I/O completion.	No
Exit	Released/terminated from executable process pool.	No

Five-state transitions to memorize

Transition	Meaning
Null -> New	Process is created.
New -> Ready	OS admits the process and is ready to handle it.
Ready -> Running	Dispatcher selects it for CPU execution.
Running -> Exit	Process finishes or is terminated.
Running -> Ready	Time slice ends or a higher-priority process becomes ready.

Running -> Blocked	Process waits for an event, such as I/O.
Blocked -> Ready	The event occurs, so it can continue when scheduled.
Ready -> Exit	OS or parent terminates it before it runs.
Blocked -> Exit	OS or parent terminates it while waiting.

Exam takeaway: Ready means waiting for CPU. Blocked means waiting for an event. This distinction is essential.

8. Queues: ready queue and blocked queues

The OS keeps processes in structures so it can track them. A queue entry usually points to the PCB of a process.

- Ready queue: processes in memory that can run when chosen.
- Blocked queue: processes waiting for an event.
- If there is only one blocked queue, the dispatcher may need to scan the whole queue when an event occurs to find who was waiting for that event.
- Multiple blocked queues improve this: each event/device can have its own queue.

Queue design	Problem / advantage
Single queue for all not-running processes	Bad because it mixes ready and blocked processes.
Ready + one blocked queue	Better, but event completion may require scanning the whole blocked queue.
Ready + multiple blocked queues	Best among these lecture options: each blocked queue can correspond to a specific event/device.

Exam takeaway: A queue contains pointers to PCBs, not necessarily the whole process.

9. Suspended processes and swapping

A suspended process has been moved out of main memory, usually to disk/secondary storage. This frees main memory.

- The processor is much faster than I/O. It is possible that all processes in memory are waiting for I/O.
- Running, Ready, and Blocked processes are normally resident in main memory.
- The OS can swap processes to disk to free memory.
- A blocked process that is swapped out becomes a suspended process.

State	Location	Waiting for CPU?	Waiting for event?
Ready	Main memory	Yes	No
Blocked	Main memory	No	Yes
Ready/Suspend	Secondary memory	Yes, but must be loaded first	No
Blocked/Suspend	Secondary memory	No	Yes

Exam takeaway: Ready/Suspend is ready logically, but it cannot run until it is swapped back into main memory.

One suspend state vs two suspend states

One suspend state is limited because it mainly allows blocked processes to be swapped out. The two-suspend-state model is better because it distinguishes a suspended process that is still waiting for an event from one that is ready to run once loaded.

Transition	Why it happens
Blocked -> Blocked/Suspend	Free memory for another process or for a ready/running process that needs memory.
Blocked/Suspend -> Ready/Suspend	The event occurs while the process is still on disk.
Ready/Suspend -> Ready	No ready process in memory, or suspended process has higher priority.
Ready -> Ready/Suspend	Need to free a large block of main memory, or swap out a lower-priority ready process.
New -> Ready or Ready/Suspend	OS creates PCB and admits to memory if possible; otherwise suspended.
Blocked/Suspend -> Blocked	Useful if a high-priority blocked process is expected to unblock soon.
Running -> Ready/Suspend	Free main memory.
Any state -> Exit	Process terminates.

Reasons for suspension

Reason	Meaning
Swapping	OS releases main memory to bring in or enlarge ready processes.
Other OS reason	OS suspects a problem such as deadlock.
Interactive user request	Debugging or user-controlled pause/background monitor.
Timing	Process runs periodically and is suspended until next time.
Parent process request	Parent suspends child/descendant to examine, modify, or coordinate it.

10. Process creation

A process can be created because a job is submitted, a user logs on, the OS needs a service process, or an existing process creates another process.

- Process spawning: a process creates another process.
- Parent process: the original creating process.
- Child process: the new process.
- Processes can form a tree, where children create more children.
- Each process is identified by a PID: process identifier.
- Parent and child may run at the same time, giving parallelism.

Creation design option	Possible choices
Resource sharing	Parent and children share all resources, share a subset, or share none.
Execution	Parent and children execute concurrently, or parent waits for child to finish.
Address space	Child duplicates parent, or child loads a different program.

Exam takeaway: Creation creates a PCB and usually places the new process into Ready or Ready/Suspend depending on memory.

UNIX fork(), exec(), wait(), exit()

System call	Meaning	Exam memory hook
fork()	Creates a new child process.	Fork = split into parent + child.
exec()	Replaces the current process memory space with a new program.	Exec = execute another program in same process.
wait()	Parent waits for child to terminate and collects status/PID.	Wait = collect child result.
exit()	Process ends and asks OS to delete it.	Exit = termination.
abort()	Parent may terminate child process.	Abort = forced termination.

Important fork() return values:

- In the parent process, fork() returns the child's PID.
- In the child process, fork() returns 0.
- If fork() fails, it returns a negative value.

Typical UNIX pattern:

```
pid = fork();
if (pid < 0)    -> fork failed
else if (pid == 0) -> child process runs exec()
else          -> parent waits using wait()
```

Exam takeaway: fork() makes a child. exec() changes the program inside a process. wait() collects child termination. exit() ends a process.

Steps of process creation

1. Assign a unique process identifier.
2. Add an entry to the primary process table.
3. Allocate space for the process: program, data, user stack, PCB.
4. Initialize the PCB.
5. Set up linkages, such as adding the process to Ready or Ready/Suspend queue.
6. Create or expand other data structures, such as accounting files.

11. Process termination

A process normally terminates when it executes its last statement and calls `exit()`. The OS then deallocates its resources. A parent may also terminate a child using `abort()`.

Termination concept	Meaning
Normal completion	Process finishes its work.
<code>exit()</code>	Process asks OS to delete it and return status to parent.
<code>wait()</code>	Parent collects status data from terminated child.
<code>abort()</code>	Parent forces child process to terminate.
Cascading termination	If parent terminates, OS may terminate all children, grandchildren, etc.

Reasons for process termination

- User logs off or normal completion.
- Batch job issues Halt.
- Time limit exceeded or event timeout.
- Memory unavailable.
- Bounds violation or protection error, such as writing to read-only file.
- Arithmetic error.
- I/O failure.
- Invalid instruction, such as trying to execute data.
- Privileged instruction or data misuse.
- OS intervention, such as deadlock handling.
- Parent terminates or parent requests termination.

Zombie vs orphan

Term	Situation	Simple meaning
Zombie	Child has terminated, but parent has not called <code>wait()</code> .	Dead process still has a record for parent to collect.
Orphan	Parent terminated without waiting for the child.	Child continues without original parent.

Exam takeaway: Zombie = child ended but parent did not collect. Orphan = parent ended while child still exists.

12. Multiprocess architecture example: Chrome

The lecture uses Chrome to show why multiple processes are useful in real applications.

Process type	Role
Browser process	Manages user interface, disk I/O, network I/O. Usually one per browser session.
Renderer process	Renders web pages and handles HTML/JavaScript. Usually new renderer per website/tab.
Plug-in process	Created for each type of plug-in.

Exam takeaway: Main advantage: isolation. If one website/tab crashes, only its renderer process is affected, not the whole browser.

13. OS control structures

To manage processes and resources, the OS keeps tables about the current status of each process and resource.

Table	What it tracks
Memory tables	Main and secondary memory allocation; protection attributes; virtual memory information.
I/O tables	Whether devices are available/assigned; I/O operation status; memory source/destination for transfer.
File tables	File existence, secondary storage location, current status, and attributes.
Process table	Where process is located in memory/storage and management attributes such as PID and status.

Exam takeaway: Tables let the OS answer: where is it, what does it own, what is it waiting for, and what can it access?

Process image

A process image is the complete picture of a process as represented in memory/storage. The lecture includes user-level, register, and system-level context.

Part	Contains
User data	Modifiable user space: program data, user stack area, and programs that may be modified.
User program / process text	Executable machine instructions.
System stack / kernel stack	Stack frames used by kernel procedures and system calls.
PCB / process table entry	Information needed by the OS to control the process.
Shared memory	Memory shared with other processes for IPC.
Register context	Program counter, processor status register, stack pointer, general-purpose registers.

14. User mode and kernel mode

Most processors have at least two execution modes. The mode controls what instructions and memory regions the code is allowed to access.

Mode	Privilege	Used by	Can do
User mode	Less privileged	Normal user programs	Regular program instructions only.
Kernel mode	More privileged	Operating system kernel	Privileged instructions, I/O, memory management, protected

How does the processor know the mode? A flag bit in the PSW (Program Status Word). It can change on interrupts, traps, and system calls.

Exam takeaway: User mode is safe/limited. Kernel mode is powerful/dangerous and used by the OS.

Functions of the OS kernel

Area	Functions
Process management	Creation/termination, scheduling/dispatching, switching, synchronization and IPC, PCB management.
Memory management	Address space allocation, swapping, page and segment management.
I/O management	Buffer management, allocation of I/O channels and devices.
Support functions	Interrupt handling, accounting, monitoring.

15. Process switching

A process switch happens when the OS stops one process and starts/resumes another. This is heavier than a simple mode switch.

When can a process switch happen?

Cause	Meaning
Clock interrupt	Running process used maximum time slice.
I/O interrupt	I/O operation completes.
Memory fault	Process references data in virtual memory; OS brings block into memory and may block current process.
Trap / exception	Error generated inside running process, such as illegal file access.
Supervisor call	Running process calls OS service, such as file open.

Process switch steps

- Save processor context, including PC and registers.
- Update the PCB of the running process: new state, reason for change, duration/accounting.
- Move the PCB to the appropriate queue: Ready, Blocked, etc.
- Select another process for execution.
- Update the selected process's PCB: state becomes Running.
- Update memory-management data structures.
- Restore context of selected process: PC, registers, etc.

Process switch vs mode switch

Concept	What changes	Cost
Process switch	CPU changes from one process to another. Must save/restore full process context and update PCBs/queues.	Higher overhead.
Mode switch	Same process continues, but CPU switches from user mode to kernel mode or back.	Lower overhead.

Exam takeaway: Every process switch includes saving/restoring context. A mode switch may only save processor state needed for the interrupt/service routine.

16. Is the OS itself a process?

The lecture gives three design approaches. You do not need to argue which is best; you need to know the difference.

Approach	How it works	Key point
Non-process kernel	Kernel executes outside any process, with its own memory and system stack.	Process concept applies only to user programs.
Execution within user processes	OS routines run in the context of the current user process after a mode switch.	Avoids extra process switches if same process resumes.
Process-based OS	Major kernel functions are organized as separate system processes.	More modular and useful for multiprocessor environments.

Unix SVR4 idea

- Most OS code executes within the environment of a user process.
- System processes run in kernel mode and execute OS code such as memory allocation and swapping.
- User processes can operate in user mode for normal code and kernel mode when executing kernel instructions.

- A user process enters kernel mode by system call, exception, or interrupt.

17. Linux process representation

In Linux, the PCB is represented by a C structure called `task_struct`.

task_struct field	Meaning
pid_t pid	Process identifier.
long state	State of the process.
unsigned int time_slice	Scheduling information.
struct task_struct *parent	Pointer to parent process.
struct list_head children	List of children.
struct files_struct *files	List of open files.
struct mm_struct *mm	Address space of this process.

- All active processes are represented using a doubly linked list of `task_struct`.
- The kernel maintains a pointer called `current` to the process currently executing.

Exam takeaway: Linux `task_struct` = Linux version of PCB.

18. UNIX process states

UNIX state	Meaning
User Running	Executing in user mode.
Kernel Running	Executing in kernel mode.
Ready to Run, in Memory	Ready to run as soon as the kernel schedules it.
Asleep in Memory	Blocked in main memory, unable to execute until an event occurs.
Ready to Run, Swapped	Ready, but must be swapped into memory before execution.
Sleeping, Swapped	Waiting for an event and swapped to secondary storage.
Preempted	Returning from kernel to user mode, but kernel schedules another process instead.
Created	Newly created and not yet ready to run.
Zombie	Process no longer exists, but leaves a record for parent to collect.

19. UNIX process image

Context level	Examples
User-level context	Process text, process data, user stack, shared memory.
Register context	Program counter, processor status register, stack pointer, general-purpose registers.
System-level context	Process table entry, U area, per-process region table, kernel stack.

UNIX process creation using `fork()`

14. Allocate a slot in the process table for the new process.
15. Assign a unique process ID to the child.
16. Copy the process image of the parent, except shared memory.
17. Increment counters for files owned by the parent because the child also owns them.
18. Assign the child to the Ready to Run state.
19. Return child's PID to the parent and 0 to the child.

After `fork()`, the dispatcher may:

- Stay in the parent process.
- Transfer control to the child process.

- Transfer control to another process while both parent and child remain Ready to Run.

20. Must memorize before the exam

Concept	One-line answer
Process	A program in execution, with state, context, and resources.
Context	All information the OS needs to manage/resume a process.
PCB	OS data structure holding process state, PC, registers, scheduling, memory, accounting, I/O info.
Ready	Can run when CPU is assigned.
Blocked	Cannot run until an event occurs.
Ready/Suspend	Ready but swapped to disk; must be loaded first.
Blocked/Suspend	Waiting for event and swapped out.
Dispatcher	Small OS program that switches CPU between processes.
fork()	Creates a new child process.
exec()	Replaces current process memory space with a new program.
wait()	Parent waits and collects child status.
exit()	Process terminates normally and OS deallocates resources.
Zombie	Child ended but parent did not wait/collect.
Orphan	Parent ended without waiting for child.
Process switch	Save old process context, update PCB/queues, select and restore new process.
Mode switch	Switch between user and kernel mode within same process; less overhead.

Most likely diagram questions

- Draw/explain five-state model: New, Ready, Running, Blocked, Exit.
- Explain why Ready and Blocked must be separate.
- Draw/explain suspend states: Ready/Suspend and Blocked/Suspend.
- Explain CPU switch from process to process using PCB save/restore.
- Explain parent/child process tree and UNIX fork behavior.

Exam takeaway: If exam panic starts: define terms clearly, then explain the transition/reason.

21. Likely exam questions with model answers

Q: Define a process.

A: A process is a program in execution. It includes the program code, associated data, current state, CPU context such as program counter and registers, and system resources.

Q: Why does the OS need a PCB?

A: The OS needs a PCB to store all information required to manage, interrupt, schedule, and later resume a process. Without the PCB, the OS would not know the process state, next instruction, registers, memory, I/O status, or scheduling information.

Q: Difference between Ready and Blocked?

A: Ready means the process can run as soon as CPU is assigned. Blocked means the process cannot run until some external event occurs, such as I/O completion.

Q: Why are multiple blocked queues useful?

A: If there is one blocked queue, the OS may scan the whole queue when an event occurs. Multiple blocked queues let the OS directly find processes waiting for a specific event/device.

Q: What is a suspended process?

A: A suspended process has been swapped out of main memory to secondary storage. It may be Ready/Suspend or Blocked/Suspend.

Q: Difference between fork() and exec()?

A: fork() creates a new child process. exec() replaces the current process memory space with a new program.

Q: What is a zombie process?

A: A process that has terminated but whose parent has not called wait() to collect its termination status.

Q: What is an orphan process?

A: A child process whose parent terminated without waiting for it.

Q: What happens during process switching?

A: The OS saves the current process context, updates its PCB and queue, selects another process, updates that process's PCB, updates memory structures, and restores the selected process context.

Q: Why is a mode switch cheaper than a process switch?

A: A mode switch changes privilege level, usually within the same process. It does not require full process selection and PCB queue changes like a process switch.

Q: Why is Chrome multiprocess?

A: To isolate websites/tabs. If one renderer crashes, other tabs and the browser process can continue.

Q: Name the OS control tables.

A: Memory tables, I/O tables, file tables, and process tables.

22. Flashcards

Front	Back
Program	Passive code on disk.
Process	Program in execution.
PID	Process identifier.
PCB	OS record/control block for a process.
Program counter	Address of next instruction to execute.
Register context	Saved CPU registers and status needed to resume.
Stack	Function calls, parameters, return addresses, local variables.
Heap	Dynamic memory allocated at runtime.
Ready	Waiting for CPU.
Blocked	Waiting for an event.
Exit	Process finished or terminated.
Dispatcher	Switches CPU from one process to another.
Timeout	Process used its time slice.
I/O interrupt	I/O operation completed.
Memory fault	Needed address/data is not currently in main memory.
Trap	Exception/error from running process.
Supervisor call	User process requests OS service.
Swapping	Moving process between main memory and disk.
Ready/Suspend	Ready but on disk.
Blocked/Suspend	Blocked and on disk.
fork()	Create child process.
exec()	Load/replace with new program.
wait()	Parent waits/collects child status.
exit()	Process terminates.
abort()	Parent kills child.
Zombie	Child dead, parent has not collected status.
Orphan	Parent gone, child still exists.
User mode	Less privileged execution mode.
Kernel mode	Privileged OS execution mode.
PSW	Program Status Word; contains status/mode/flags.
task_struct	Linux structure representing a process/PCB.

Final 10-minute checklist

- Can I define process, program, context, PCB?
- Can I explain text, data, stack, heap?
- Can I draw five-state model and explain every arrow?
- Can I explain Ready vs Blocked vs Suspended?
- Can I explain fork(), exec(), wait(), exit()?
- Can I explain zombie vs orphan?
- Can I list process switch steps?
- Can I compare process switch and mode switch?
- Can I explain user mode vs kernel mode?
- Can I name Linux task_struct and UNIX process states?

Exam takeaway: Do not over-study tiny details if time is short. The exam will most likely reward clean definitions and clear state-transition explanations.

23. Professor edge-case addendum

Use this section after you know the main diagrams. These are the small technical details that can appear as direct theory questions or diagram-label questions.

23.1 Program Status Word (PSW) and x86 EFLAGS

Core idea: A PSW is a processor register that stores condition codes plus other status/control information. The lecture uses the x86 EFLAGS register as the example.

- ZF = Zero Flag: important status flag. It is set when the result of an operation is zero.
- CF = Carry Flag: important status flag. It records carry/borrow, especially for unsigned arithmetic.
- DF = Direction Flag: control flag. It controls the direction of string operations.
- IF = Interrupt Enable Flag: system flag in the lecture table. It controls whether maskable interrupts are enabled.
- Other flags shown include OF, SF, AF, PF, TF, IOPL, NT, RF, VM, AC, VIF, VIP, and ID.

Flag	Name	Fast exam meaning
ZF	Zero flag	Result is zero.
CF	Carry flag	Carry/borrow occurred; useful for unsigned arithmetic.
DF	Direction flag	Direction of string processing.
IF	Interrupt enable flag	Whether interrupts are enabled.
OF	Overflow flag	Signed overflow occurred.
SF	Sign flag	Result sign bit is set.
PF	Parity flag	Parity of low byte.
AF	Auxiliary carry flag	Carry from lower nibble; used in BCD-related operations.

23.2 UNIX Process Table Entry vs UNIX U Area

The distinction to memorize: the UNIX process table entry stores information always accessible to the OS; the U area stores process-control information accessed only in the context of that process.

Structure	What it means	Fields to recognize
UNIX Process Table Entry	Always accessible to the OS. Defines the current process state and information needed for scheduling, memory, signaling, and swapping.	Process status; pointers to U area and process memory area (text, data, stack); process size; user identifiers; process identifiers; event descriptor; priority; signal; timers; P_link; memory status.
UNIX U Area	Process-control information used only when the process context is active.	Process table pointer; user identifiers; timers; signal-handler array; control terminal; error field; return value; I/O parameters; file parameters; user file descriptor table; limit fields; permission modes fields.

Memory trick: Process Table Entry = always visible control summary. U Area = user/process-context details such as signals, terminals, open-file table, syscall errors and return values.

23.3 PCB vulnerability and protection

The PCB is dangerous because it is central. The lecture calls it the most important data structure in the OS. It is read and modified by OS modules such as scheduling, interrupt processing, and performance monitoring.

- The PCB defines the state of the OS. If PCBs are corrupted, the OS may lose the ability to manage processes correctly.
- Direct access from many OS routines is risky: one bug in one routine could damage PCBs and crash process management.
- A PCB design change can affect many OS modules because many modules depend on its structure.
- Protection solution: force OS routines to access PCBs through a handler routine.
- Tradeoff: the handler protects PCBs, but it can add performance overhead.

23.4 Process list structures: visual logic

Diagram rule: queues are usually drawn as linked lists of pointers to PCBs. A queue entry does not need to contain the whole process; it can point to the process PCB in memory.

Ready queue head -> PCB_A -> PCB_D -> PCB_F -> NULL

Blocked on disk I/O -> PCB_B -> PCB_C -> NULL

Blocked on keyboard -> PCB_E -> NULL

- Each PCB contains process information such as state, priority, program counter, registers, memory pointers, I/O status, and accounting information.
- A Process Table Entry may include P_link, which points to the next process in the ready queue.
- Separate blocked queues are useful because each event/device can have its own waiting list, so the OS does not need to scan one huge blocked queue.

23.5 Rapid recall for these edge cases

Q: Which x86 register is used as the PSW example? **A:** EFLAGS.

Q: Which flag means the result is zero? **A:** ZF, the Zero Flag.

Q: Which flag controls direction of string operations? **A:** DF, the Direction Flag.

Q: Which UNIX structure is always accessible to the OS? **A:** The UNIX Process Table Entry.

Q: Where is the signal-handler array? **A:** In the UNIX U Area.

Q: Where is the user file descriptor table? **A:** In the UNIX U Area.

Q: What does P_link do? **A:** It points to the next process in the ready queue.

Q: Why is direct PCB access dangerous? **A:** A bug in one OS routine can damage PCBs and destroy process management.

Q: What protects PCBs in the proposed design? **A:** A handler routine that all OS routines must use to access PCBs.

Q: What is the cost of using a PCB handler routine? **A:** Performance overhead.

Extra final 5-minute checklist

- Can I recognize ZF, CF, DF, IF in the x86 EFLAGS/PSW diagram?
- Can I separate UNIX Process Table Entry fields from UNIX U Area fields?
- Can I explain why direct PCB access is dangerous and why a handler routine helps?
- Can I draw ready/blocked queues as linked lists pointing to PCBs?
- Can I explain P_link as the next-link pointer in the ready queue?

Source: Lecture 3 - Processes, Dr. Noha Adly, Operating Systems - CS x61. Updated with PSW/EFLAGS, UNIX Process Table Entry, UNIX U Area, PCB protection, and process-list edge cases.